NLP Question Bank — PART A Answers

Answer ANY FIVE Questions (5 × 20 = 100 Marks)

Question 1.a: Analyze the role of NLP in Sentiment Analysis and its impact on social media monitoring (10 Marks)

Introduction

Sentiment Analysis (SA) identifies and quantifies subjective opinions (positive, negative, neutral) from text data using NLP techniques.

Role of NLP in Sentiment Analysis

1. Text Preprocessing Pipeline

- Tokenization → Stop word removal → Normalization → Lemmatization/Stemming
- Example: "I'm loving it!" → ["love", "it"]

2. Feature Extraction

- **BoW/TF-IDF**: Convert text to numerical vectors
- **Word Embeddings**: Word2Vec, GloVe, BERT for semantic meaning
- **N-grams**: Capture context ("not good" ≠ "good")

3. Classification Methods

- **Lexicon-based**: VADER, SentiWordNet (dictionary lookup)
- **ML**: Naive Bayes, SVM, Logistic Regression
- **Deep Learning**: LSTM, CNN, Transformer (BERT, GPT)

Impact on Social Media Monitoring

1. Brand Reputation Management

- **Real-time monitoring** of customer opinions on Twitter, Facebook, Instagram
- Early detection of PR crises or negative sentiment spikes
- Example: Companies can quickly respond to product complaints trending on social media

2. Customer Insights & Feedback Analysis

- Understanding customer pain points and preferences
- Product improvement based on user reviews
- Identifying feature requests and common complaints

**Real-time tracking of customer opinions across platforms

- Early crisis detection via negative sentiment spikes
- Automated alert systems for brand mentions

2. Customer Insights

- Analyze product reviews for pain points
- Identify feature requests and improvement areas
- Track customer satisfaction trends

3. Market Research

- Competitor sentiment comparison
- Trend identification and prediction
- Consumer behavior analysis

4. Campaign Effectiveness

- Measure marketing ROI via sentiment scores
- A/B testing message effectiveness
- Influencer impact measurement

Key Challenges

- **Sarcasm/Irony**: "Great job!" (may be negative)
- **Context**: "This phone is sick!" (slang = positive)
- **Multilingual**: Code-mixing, regional languages
- **Ambiguity**: "cheap" (positive cost vs negative quality)

Conclusion

NLP enables scalable, automated sentiment analysis of millions of social media posts, empowering data-driven decisions, proactive customer engagement, and brand protection in real-time

1. Tokenization

Definition: The process of splitting text into smaller units called **tokens** (words, subwords, or characters).

Types:

- **Word Tokenization**: "I love NLP" → ["I", "love", "NLP"]
- **Sentence Tokenization**: Splitting paragraphs into sentences
- **Subword Tokenization**: "unbelievable" → ["un", "believ", "able"] (used in BERT)

Purpose:

- Converts continuous text into discrete units
- Foundation for all downstream NLP tasks
- Enables counting, indexing, and vectorization

Example:

```
from nltk.tokenize import word_tokenize
text = "NLP is amazing! Isn't it?"
```

```
tokens = word_tokenize(text)
# Output: ['NLP', 'is', 'amazing', '!', 'Is', "n't", 'it', '?']
```

Challenges:

- Handling punctuation and special characters
- Contractions ("don't" → "do" + "n't")
- Language-specific rules (spaces in English vs no spaces in Chinese)

---

2. Stemming

Definition: The process of reducing words to their **root/base form** by removing suffixes using heuristic rules (often crude chopping).

Algorithm: Porter Stemmer, Snowball Stemmer, Lancaster Stemmer

Purpose:

- Reduce vocabulary size -okens = word_tokenize("NLP is amazing!")

['NLP', 'is', 'amazing', '!']

```
| studies | studi |
| studying | studi |
| connection | connect |
| connects | connect |
| connected | connect |

**Python Example**:
```python
from nltk.stem import PorterStemmer
stemmer = PorterStemmer()
words = ["running", "runs", "runner", "easily", "fairly"]
stems = [stemmer.stem(word) for word in words]
# Output: ['run', 'run', 'runner', 'easili', 'fairli']
```
```

Advantages:

- Very fast (rule-based)
- Language-agnostic (with appropriate rules)
- Reduces dimensionality

Disadvantages:

- **Not always linguistically correct**: "studies" → "studi" (not a real word)
- **Over-stemming**: "universal" and "university" → "univers"
- Example**:

```
from nltk.stem import PorterStemmer
stemmer = PorterStemmer()
stemmer.stem("running")  # 'run'
stemmer.stem("studies")  # 'studi' (not a real word!)
```

Pros: Fast, reduces vocabulary size
Cons: Produces invalid words, over/under-stemm | running | run | | ran | run | | better | good | | studies | study | | studying | study | | is, am, are | be | | geese | goose | | cacti | cactus |

Python Example:

```
from nltk.stem import WordNetLemmatizer
from nltk.corpus import wordnet

lemmatizer = WordNetLemmatizer()

# Without POS tag (assumes noun)
print(lemmatizer.lemmatize("running"))  # running

# With POS tag (verb)
print(lemmatizer.lemmatize("running", pos='v'))  # run
print(lemmatizer.lemmatize("better", pos='a'))   # good
```

Advantages:

- Returns real dictionary words
- Linguistically accurate
- Context-aware (with POS tagging)
- Better for semantic analysis

Disadvantages:

- Slower than stemming (requires dictionary lookup)
- Requires POS tagging for best results
- Language-specific dictionaries needed

--Example**:

```
from nltk.stem import WordNetLemmatizer
lemmatizer = WordNetLemmatizer()
lemmatizer.lemmatize("running", pos='v')  # 'run'
lemmatizer.lemmatize("better", pos='a')   # 'good'
```

Pros: Linguistically correct, preserves meaning
Cons: Slower, needs POS tags for accuracy

- When speed is critical (large-scale IR systems)

- When exact word forms don't matter
- Search engines (query expansion)
- Document clustering

Use Lemmatization:

- When linguistic accuracy is important
- Sentiment analysis (preserving meaning)
- Question answering systems
- Machine translation
- Text summarization

Example Pipeline Comparison

Original: "The students are studying better techniques for running faster."

After Tokenization:

```
['The', 'students', 'are', 'studying', 'better', 'techniques', 'for',
'running', 'faster']
```

After Stemming:

```
['the', 'student', 'are', 'studi', 'better', 'techniqu', 'for', 'run',
'faster']
```

After Lemmatization (with POS):

```
['the', 'student', 'be', 'study', 'good', 'technique', 'for', 'run',
'fast']
```

Conclusion

All three techniques serve complementary roles in text preprocessing. **Tokenization** is mandatory and comes first. The choice between **stemming** (fast, crude) and **lemmatization** (accurate, slower) depends on the application requirements—stemming for speed-critical IR tasks, lemmatization for semantic-rich NLU tasks.

##Tokenization**: Always first step in any pipeline
Stemming: Speed-critical tasks (IR, search), clustering
Lemmatization: Accuracy-critical tasks (QA, sentiment, translation)

Pipeline Comparison

```
Original:     "The students are studying"
Tokenization: ['The', 'students', 'are', 'studying']
Stemming:     ['the', 'student', 'are', 'studi']
Lemmatization:['the', 'student', 'be', 'study']
```

Conclusion

Tokenization splits text into units (mandatory first step). **Stemming** is fast but crude. **Lemmatization** is accurate but slower. Choose based on speed vs accuracy trade-off

- Understanding sentence structure and grammar

5. Machine Translation

- Preserving grammatical structure in target language
- Proper word order in translation

6. Text-to-Speech (TTS)

- Pronunciation depends on POS
- "live" (verb) /lɪv/ vs "live" (adjective) /laɪv/

7. Question Answering

- Understanding question type based on interrogative words and POS
- "What" (WP) expects noun phrase answer

8. Sentiment Analysis

- Adjectives and adverbs carry sentiment
- "The movie is **extremely boring**" (RB JJ)

Methods for POS Tagging

1. Rule-Based Tagging

Approach: Uses hand-crafted linguistic rules and dictionaries.

Method:

- Lexical rules: "Words ending in '-ly' are adverbs"
- Contextual rules: "Word after 'the' is likely a noun or adjective"

Example:

```
Rule: DT + JJ* + NN
"The beautiful house" → DT JJ NN
```

Advantages:

- Interpretable and transparent
- Can incorporate linguistic knowledge

Disadvantages:

- Labor-intensive to create rules
- Cannot handle ambiguity well
- Language-specific

---

2. Stochastic/Probabilistic Tagging

Uses statistical models trained on annotated corpora.

a) Hidden Markov Model (HMM)

Concept:

- POS tags are hidden states
- Words are observations
- Find most likely sequence of tags given words

Components:

1. **Transition Probability**: P(tag_i | tag_{i-1})
    - "What's the probability of a noun after a determiner?"
2. **Emission Probability**: P(word | tag)
    - "What's the probability of seeing 'run' given it's a verb?"

Algorithm: Viterbi algorithm for decoding

Formula:

```
Best Tag Sequence = argmax P(tags | words)
                  = argmax P(words | tags) × P(tags)
```

Example:

```
Sentence: "I love NLP"
States (tags): [PRON, VERB, NOUN]
Observations: [I, love, NLP]
- Hand-crafted linguistic rules: "Words ending in '-ly' → adverb"
- Contextual rules: "After 'the' → noun/adjective"
- **Pros**: Interpretable | **Cons**: Labor-intensive, poor ambiguity
handling

### 2. Hidden Markov Model (HMM)
**Concept**: Find most likely tag sequence using:
- **Transition probability**: P(tag_i | tag_{i-1})
- **Emission probability**: P(word | tag)
```

```
- **Algorithm**: Viterbi decoding

**Example**: "I love NLP" → [PRON, VERB, NOUN]

**Pros**: Fast, efficient | **Cons**: Limited context (only previous tag)

### 3. Transformation-Based (Brill Tagger)
- Start with most frequent tags
- Apply learned correction rules: "Change NN → VB if previous word is
'to'"
- **Pros**: Interpretable, combines rule + statistics

### 4. Neural Network Approaches

**a) LSTM/BiLSTM**
```

Word Embeddings → BiLSTM → Softmax → Tags

```
- **Pros**: Long-range context, better than HMM

**b) Transformer (BERT)**
```python
# Fine-tune pre-trained BERT for POS tagging
from transformers import BertForTokenClassification
model = BertForTokenClassification.from_pretrained('bert-base')
```
```

- **Pros**: 97%+ accuracy, handles OOV words | **Cons**: Computationally expensiv

Question 2.b: Differentiate between Bag of Words and Continuous Bag of Words (10 Marks)

Introduction

Both **Bag of Words (BoW)** and **Continuous Bag of Words (CBOW)** are methods for representing text as numerical vectors, but they serve different purposes and operate on different principles. BoW is a classical feature extraction technique, while CBOW is a neural network architecture for learning word embeddings.

---

1. Bag of Words (BoW)

Definition: A representation technique that converts text into a fixed-length vector based on word frequency, **ignoring grammar and word order**.

How It Works

Step 1: Build Vocabulary Collect all unique words from the corpus.

Step 2: Create Vector For each document, count the occurrences of each word from the vocabulary.

Example

Corpus:

```
Doc 1: "I love NLP"
Doc 2: "I love machine learning"
Doc 3: "NLP is great"
```

Vocabulary: [I, love, NLP, machine, learning, is, great]

BoW Representation:

```
       I  love  NLP  machine  learning  is  great
Doc1: [1,   1,   1,     0,       0,     0,   0]
Doc2: [1,   1,   0,     1,       1,     0,   0]
Doc3: [0,   0,   1,     0,       0,     1,   1]
```

Python Implementation

```python
from sklearn.feature_extraction.text import CountVectorizer

corpus = [
    "I love NLP",
    "I love machine learning",
    "NLP is great"
]

vectorizer = CountVectorizer()
bow_matrix = vectorizer.fit_transform(corpus)

print("Vocabulary:", vectorizer.get_feature_names_out())
print("BoW Matrix:\n", bow_matrix.toarray())
```

Variants of BoW

1. Binary BoW: 1 if word present, 0 if absent (ignores frequency)

```
Doc1: [1, 1, 1, 0, 0, 0, 0]
```

2. TF-IDF (Term Frequency-Inverse Document Frequency): Weights words by importance (rare words across corpus get higher weight)

```
TF-IDF(word) = TF(word, doc) × log(N / DF(word))
```

Characteristics of BoW

Advantages:

- Simple and easy to implement
- Interpretable (each dimension = specific word)
- Works well for text classification
- Language-agnostic

Disadvantages:

- **Ignores word order**: "not good" = "good not"
- **Ignores semantics**: "car" and "automobile" are different dimensions
- **High dimensionality**: Vocabulary size = vector size (can be 10,000+)
- **Sparse vectors**: Most values are 0
- **No context**: "bank" (river) vs "bank" (financial) treated identically

Use Cases

- Document classification
- Spam detection
- Topic modeling
- Information retrieval (simple search)

---

2. Continuous Bag of Words (CBOW)

Definition: A neural network architecture used to learn **dense word embeddings** by predicting a target word from its surrounding context words.

Part of: Word2Vec model (Mikolov et al., 2013)

How It Works

Objective: Given context words, predict the center/target word.

Architecture:

```
Input: Context words (one-hot encoded)
       ↓
Hidden Layer (Projection/Average)
       ↓
Output: Target word (softmax over vocabulary)
```

Example

Sentence: "The quick brown fox jumps"

Training Sample (window size = 2):

```
Context: [The, quick, fox, jumps] → Target: brown
Context: [quick, brown, jumps, over] → Target: fox
```

Process:

1. One-hot encode context words
2. Look up embeddings for each context word
3. Average the context word embeddings
4. Pass through output layer (softmax)
5. Predict target word
6. Update embeddings via backpropagation

Mathematical Formulation

Input: One-hot vectors of C context words: $\mathbf{x}_1, \mathbf{x}_2, ..., \mathbf{x}_C$

Hidden Layer: Average of context word embeddings $$ \mathbf{h} = \frac{1}{C} \sum_{i=1}^{C} \mathbf{W}^T \mathbf{x}_i $$

Output: Softmax over vocabulary $$ P(w_O | w_{I,1}, ..., w_{I,C}) = \frac{\exp(\mathbf{v}*{w_O}^T |mathbf{h})}{|sum*{w=1}^{V} \exp(\mathbf{v}_w^T \mathbf{h})} $$

Loss: Cross-entropy loss (maximize probability of correct target word)

Python Implementation

```python
from gensim.models import Word2Vec

# Sample corpus
sentences = [
    ['I', 'love', 'natural', 'language', 'processing'],
    ['machine', 'learning', 'is', 'fascinating'],
    ['deep', 'learning', 'and', 'NLP', 'are', 'related']
]

# Train CBOW model
model = Word2Vec(sentences,
                 sg=0,           # 0 = CBOW, 1 = Skip-gram
                 vector_size=100, # Embedding dimension
                 window=5,        # Context window size
                 min_count=1,     # Minimum word frequency
                 workers=4)       # Parallel processing

# Get word vector
vector = model.wv['learning']
print("Vector for 'learning':", vector[:10])  # First 10 dimensions
```

```
# Find similar words
similar = model.wv.most_similar('learning', topn=3)
print("Similar to 'learning':", similar)
```

Characteristics of CBOW

Advantages:

- **Dense representations**: Typically 100-300 dimensions (vs 10,000+ for BoW)
- **Semantic similarity**: Similar words have similar vectors
  - `king - man + woman ≈ queen`
- **Efficient**: Faster to train than Skip-gram
- **Captures context**: Word meaning based on surrounding words

Disadvantages:

- Requires large corpus for training
- No interpretability (dimensions are latent features)
- Fixed vocabulary (can't handle OOV words without tricks)
- Loses word order information

Use Cases

- Pre-trained embeddings for downstream tasks
- Semantic similarity computation

```
Corpus: ["I love NLP", "I love ML", "NLP is great"]
Vocabulary: [I, love, NLP, ML, is, great]

BoW Representation:
Doc1: [1, 1, 1, 0, 0, 0]
Doc2: [1, 1, 0, 1, 0, 0]
Doc3: [0, 0, 1, 0, 1, 1]
```

Python Snippet

```
from sklearn.feature_extraction.text import CountVectorizer
vectorizer = CountVectorizer()
bow_matrix = vectorizer.fit_transform(corpus)
```

Variants

- **Binary BoW**: 1/0 (present/absent)
- **TF-IDF**: Weight by rarity → TF × log(N/DF)

Characteristics

Pros: Simple, interpretable, language-agnostic
Cons: Ignores order/semantics, high-dimensional, sparse

Use Cases: Text classification, spam detection, IRss of extracting structured data from websites. **Beautiful Soup** is a popular Python library that parses HTML/XML documents and provides a simple API for navigating, searching, and modifying the parse tree.

Why Beautiful Soup?

- Easy to learn and use
- Handles messy/malformed HTML gracefully
- Powerful search methods (CSS selectors, regex, functions)
- Works with different parsers (html.parser, lxml, html5lib)

Installation

```
pip install beautifulsoup4
pip install requests  # For fetching web pages
pip install lxml      # Fast parser (optional but recommended)
```

Basic Process of Web Scraping

Step 1: Fetch HTML content using `requests`
Step 2: Parse HTML with Beautiful Soup
Step 3: Navigate and search the parse tree
Step 4: Extract desired data
Step 5: Clean and store data

---

1. Fetching HTML Content

```python
import requests
from bs4 import BeautifulSoup

# Fetch webpage
url = "https://example.com"
response = requests.get(url)

# Check if request was successful
if response.status_code == 200:
    html_content = response.text
else:
    print(f"Failed to retrieve page: {response.status_code}")
```

Important Headers (to avoid being blocked): **Objective**: Predict center word from context words

Architecture: Context → Average embeddings → Softmax → Target word

Example

```
Sentence: "The quick brown fox jumps"
Context: [The, quick, fox, jumps] → Target: brown
```

Training Process

1. One-hot encode context words
2. Average their embeddings: $h = \frac{1}{C}\sum W^T x_i$
3. Softmax: $P(w_O|context) = \frac{e^{v_{w_O}^T h}}{\sum e^{v_w^T h}}$
4. Backprop to update embeddings

Python Snippet

```python
from gensim.models import Word2Vec
sentences = [['I', 'love', 'NLP'], ['ML', 'is', 'great']]
model = Word2Vec(sentences, sg=0, vector_size=100, window=5)
vector = model.wv['NLP']  # Get 100-dim embedding
similar = model.wv.most_similar('NLP')  # Find similar words
```

Characteristics

Pros: Dense (100-300 dims), semantic similarity (king-man+woman≈queen), efficient
Cons: Needs large corpus, no interpretability, fixed vocab

Use Cases: Pre-trained embeddings, similarity, downstream task

```python
import re

# Find tags with class starting with 'bo'
books = soup.find_all('div', class_=re.compile(r'^bo'))

# Find tags by attribute pattern
elements = soup.find_all(id=re.compile(r'book\d+'))
```

---

4. Extracting Data

a) Extracting Text

```
# Get text content (strips HTML tags)
title = soup.find('h2', class_='title')
print(title.text)           # Output: Python for Beginners
print(title.get_text())     # Same as above
print(title.string)         # NavigableString object
```

b) Extracting Attributes

```
# Get attribute value
book_div = soup.find('div', class_='book')
book_id = book_div['id']           # Output: book1
book_id = book_div.get('id')       # Safer (returns None if not found)
book_class = book_div.get('class') # Output: ['book']
```

c) Navigating Relatives

```
# Parent
title = soup.find('h2', class_='title')
parent_div = title.parent

# Siblings
first_book = soup.find('div', id='book1')
next_sibling = first_book.find_next_sibling('div')

# Children
book_div = soup.find('div', class_='book')
children = book_div.findChildren()
```

---

5. Complete Example: Scraping Book Data

```
import requests
from bs4 import BeautifulSoup
import pandas as pd

# Fetch page
url = "https://books.toscrape.com/"
response = requests.get(url)
soup = BeautifulSoup(response.content, 'lxml')

# Initialize lists to store data
titles = []
prices = []
ratings = []
availability = []
```

```python
# Find all book containers
books = soup.find_all('article', class_='product_pod')

for book in books:
    # Extract title
    title = book.h3.a['title']
    titles.append(title)

    # Extract price
    price = book.find('p', class_='price_color').text
    prices.append(price)

    # Extract rating
    rating_class = book.find('p', class_='star-rating')['class'][1]
    ratings.append(rating_class)

    # Extract availability
    avail = book.find('p', class_='instock availability').text.strip()
    availability.append(avail)

# Create DataFrame
df = pd.DataFrame({
    'Title': titles,
    'Price': prices,
    'Rating': ratings,
    'Availability': availability
})

print(df.head())

# Save to CSV
df.to_csv('books_data.csv', index=False)
```

---

6. Advanced Techniques

a) Handling Pagination

```python
base_url = "https://example.com/page="
all_data = []

for page in range(1, 6):  # Scrape 5 pages
    url = f"{base_url}{page}"
    response = requests.get(url)
    soup = BeautifulSoup(response.content, 'lxml')

    # Extract data from this page
    items = soup.find_all('div', class_='item')
    all_data.extend(extract_data(items))  # Your extraction function
```

b) Handling JavaScript-Rendered Content

Beautiful Soup can't execute JavaScript. For dynamic content, use **Selenium** instead:

```
from selenium import webdriver

driver = webdriver.Chrome()
driver.get(url)
html = driver.page_source
soup = BeautifulSoup(html, 'lxml')
# Now scrape as usual
driver.quit()
```

c) Error Handling

Basic Process

1. Fetch HTML → 2. Parse → 3. Search/Navigate → 4. Extract → 5. Store

Setup

```
import requests
from bs4 import BeautifulSoup

# Fetch webpage
url = "https://example.com"
headers = {'User-Agent': 'Mozilla/5.0'}  # Avoid blocking
response = requests.get(url, headers=headers)
soup = BeautifulSoup(response.content, 'lxml'

**3. Check Terms of Service**
- Some websites prohibit scraping
- Always check legal terms

**4. Use APIs when available**
- Twitter API, Reddit API > scraping

**5. Handle errors gracefully**
```python
try:
    data = element.find('span').text
except (AttributeError, TypeError):
    data = "N/A"
```
```

6. User-Agent

```
headers = {'User-Agent': 'Your Bot Name (contact@email.com)'}
```

---

9. Common Challenges & Solutions

Challenge	Solution
Dynamic content (JS)	Use Selenium/Playwright
Rate limiting	Add delays, use proxies
CAPTCHAs	CAPTCHA solving services (expensive)
IP blocking	Rotate IPs, use proxy services
Changing structure	Robust selectors, regular updates
Malformed HTML	Beautiful Soup handles it well

Conclusion

Beautiful Soup is a powerful, beginner-friendly library for web scraping. Combined with `requests` for fetching pages and `pandas` for data storage, it forms a complete scraping toolkit. The key is to understand HTML structure, use appropriate selectors (CSS selectors are most flexible), and always scrape ethically and responsibly.

Question 3.b: Illustrate the concept of document similarity using appropriate algorithms and discuss its importance in search engines (10 Marks)

Introduction

Navigation & Extraction Methods

Finding Elements

```
# By tag
soup.find('h1')              # First <h1>
soup.find_all('div')         # All <div>

# By class/ID
soup.find('h2', class_='title')
soup.find('div', id='book1')

# CSS Selectors (most powerful)
soup.select('.book .title')  # Class selector
soup.select_one('#book1')    # ID selector
soup.select('div > p')       # Direct child
```

Extracting Data

```
# Text
title = soup.find('h2', class_='title')
title.text  # or title.get_text()

# Attributes
book_div = soup.find('div', class_='book')
book_id = book_div['id']  # or book_div.get('id')

# Navigation
parent = title.parent
sibling = book_div.find_next_sibling('div'

    return len(intersection) / len(union)

doc1 = "AI is the future"
doc2 = "AI and ML are the future"

similarity = jaccard_similarity(doc1, doc2)
print(f"Jaccard Similarity: {similarity:.2f}")
# Output: 0.43  (3 common words / 7 total unique words)
```

Advantages:

- Simple and intuitive
- Good for short texts

Disadvantages:

- Treats all words equally (no weighting)
- Ignores word frequency

---

3. Euclidean Distance

Concept: Measures straight-line distance between two vectors in n-dimensional space. **Smaller = more similar**.

$$ d(A, B) = \sqrt{\sum_{i=1}^{n} (A_i - B_i)^2} $$

Python Implementation

```
from scipy.spatial import distance

vec1 = tfidf_matrix[0].toarray()[0]
vec2 = tfidf_matrix[1].toarray()[0]

euclidean_dist = distance.euclidean(vec1, vec2)
print(f"Euclidean Distance: {euclidean_dist:.4f}")
```

```
# Convert to similarity (0 to 1 scale)
similarity = 1 / (1 + euclidean_dist)
```

Disadvantages:

- Sensitive to document length
- Less common than cosine for text

---

4. Word Embeddings-Based Similarity

Concept: Represent documents as average of word embeddings, then compute cosine similarity.

Using Word2Vec

```
from gensim.models import Word2Vec
import numpy as np

# Train Word2Vec
sentences = [doc.split() for doc in documents]
model = Word2Vec(sentences, vector_size=100, window=5, min_count=1)

def document_vector(doc, model):
    # Average word vectors
    vectors = [model.wv[word] for word in doc.split() if word in model.wv]
    if len(vectors) == 0:
        return np.zeros(model.vector_size)
    return np.mean(vectors, axis=0)

# Get document vectors
doc1_vec = document_vector(documents[0], model)
doc2_vec = document_vector(documents[1], model)

# Compute similarity
similarity = np.dot(doc1_vec, doc2_vec) / (np.linalg.norm(doc1_vec) *
np.linalg.norm(doc2_vec))
print(f"Semantic Similarity: {similarity:.4f}")
```

Advantages:

- Captures semantic similarity
- "car" and "automobile" are close

Disadvantages:

- Requires pre-trained embeddings
- Simple averaging loses context

Complete Example: Scraping Books

```python
import requests
from bs4 import BeautifulSoup
import pandas as pd

url = "https://books.toscrape.com/"
response = requests.get(url)
soup = BeautifulSoup(response.content, 'lxml')

titles, prices = [], []
books = soup.find_all('article', class_='product_pod')

for book in books:
    title = book.h3.a['title']
    price = book.find('p', class_='price_color').text
    titles.append(title)
    prices.append(price)

df = pd.DataFrame({'Title': titles, 'Price': prices})
df.to_csv('books.csv', index=False)
```

Advanced Techniques

Pagination

```python
for page in range(1, 6):
    url = f"https://example.com/page={page}"
    # Scrape each page
    time.sleep(1)  # Rate limiting
```

Error Handling

```python
try:
    price = book.find('span', class_='price').text
except AttributeError:
    price = None
```

JavaScript Content: Use Selenium for dynamic sites

```python
from selenium import webdriver
driver = webdriver.Chrome()
driver.get(url)
soup = BeautifulSoup(driver.page_source, 'lxml')
```

Best Practices

1. **Respect robots.txt**: Check allowed/disallowed paths
2. **Rate limiting**: Use `time.sleep()` between requests
3. **User-Agent**: Set proper headers
4. **Error handling**: Use try-except for missing elements
5. **APIs first**: Use official APIs when available
6. **Legal compliance**: Check ToS before scraping

Conclusion

Beautiful Soup + requests provides a powerful scraping toolkit. Use CSS selectors for flexible element selection, handle errors gracefully, and always scrape ethicalFormula $$ \text{cosine}(A, B) = \frac{A \cdot B}{||A|| \times ||B||} = \frac{\sum A_i B_i}{\sqrt{\sum A_i^2} \times \sqrt{\sum B_i^2}} $$

Example

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

docs = ["AI is the future", "AI and ML are the future", "The past"]
vectorizer = TfidfVectorizer()
tfidf_matrix = vectorizer.fit_transform(docs)
similarity_matrix = cosine_similarity(tfidf_matrix)
# [[1.   0.65 0.32]
#  [0.65 1.   0.28]
#  [0.32 0.28 1.  ]]
```

Pros: Fast, normalized, sparse-friendly
Cons: Ignores word order, no semantics### 2. Jaccard Similarity **Formula**: $J(A, B) = \frac{|A \cap B|}{|A \cup B|}$

```
def jaccard(doc1, doc2):
    set1, set2 = set(doc1.split()), set(doc2.split())
    return len(set1 & set2) / len(set1 | set2)

# "AI is future" vs "AI and ML are the future"
# 3 common / 7 total = 0.43
```

Pros: Simple, intuitive | **Cons**: No weighting, ignores frequency

3. Euclidean Distance

$d(A, B) = \sqrt{\sum (A_i - B_i)^2}$ (smaller = more similar)

Cons: Sensitive to length, less common### 4. Word2Vec Average

```
from gensim.models import Word2Vec
model = Word2Vec(sentences, vector_size=100)
doc_vec = np.mean([model.wv[w] for w in doc.split()], axis=0)
# Then cosine similarity
```

Pros: Semantic similarity | **Cons**: Averaging loses context

5. Doc2Vec

```
from gensim.models.doc2vec import Doc2Vec, TaggedDocument
tagged = [TaggedDocument(doc.split(), [i]) for i, doc in enumerate(docs)]
model = Doc2Vec(tagged, vector_size=100, epochs=40)
vec = model.infer_vector(doc.split())
```

Pros: Document-level embeddings

6. BERT (Transformer)

```
from sentence_transformers import SentenceTransformer, util
model = SentenceTransformer('all-MiniLM-L6-v2')
embeddings = model.encode(docs)
similarity = util.cos_sim(embeddings[0], embeddings[1])
```

Pros: Best semantic accuracy, contextual | **Cons**: Expensive, needs GPU### Application in Search Engines

Search Pipeline

```
Query → Vector → Candidate Retrieval (TF-IDF) → Re-ranking (BERT) →
Results
```

1. Indexing: Pre-compute TF-IDF for all documents
2. Query Matching: Convert query to vector, compute cosine similarity
3. Top-K Retrieval: Get top 1000 candidates (fast)
4. Neural Re-ranking: BERT scores top candidates (accurate)

"More Like This"

```
def find_similar(doc_id, top_n=5):
    sims = cosine_similarity(tfidf_matrix[doc_id], tfidf_matrix)[0]
    return sims.argsort()[-top_n-1:-1][::-1]
```

Duplicate Detection

```
# Find pairs with similarity > 0.95
duplicates = [(i,j) for i in range(n) for j in range(i+1,n)
              if similarity_matrix[i][j] > 0.95]
```

Optimizations

- **Inverted Index**: Fast doc lookup
- **Caching**: Store popular queries
- **LSH**: Approximate similarity for scale
- **Hybrid**: TF-IDF (retrieval) + BERT (re-ranking)

Conclusion

Document similarity powers search engines. Classical methods (TF-IDF + cosine) provide fast retrieval; neural models (BERT) enable semantic understanding. Modern systems use hybrid approaches: fast retrieval + accurate re-